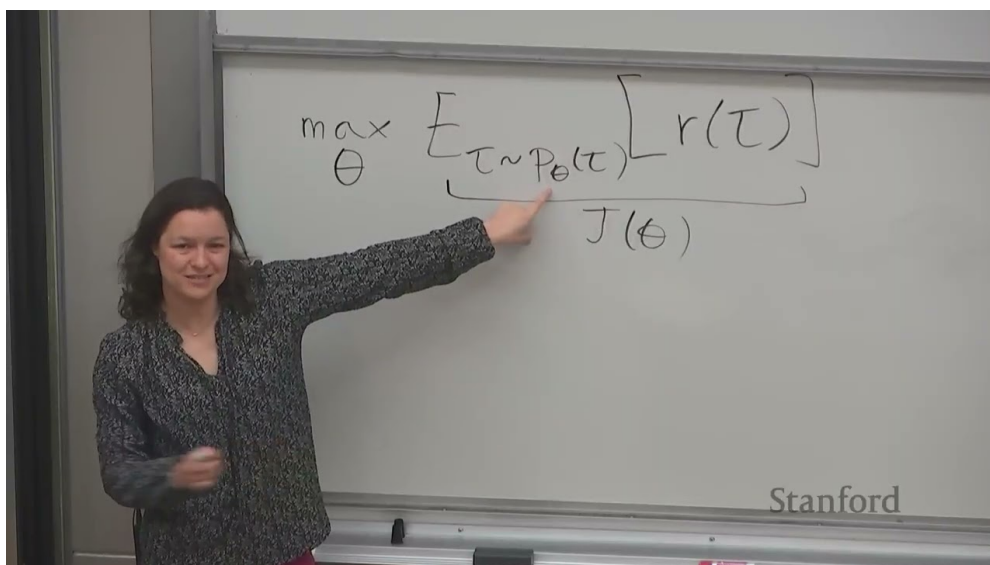


课程笔记

## CS224R Lecture 3: 策略梯度

基于公开课程资料整理

2025 年春季



视频作者/频道: Stanford Online

发布日期: 2025-04

视频时长: 约 80 分钟

讲者: Chelsea Finn

课程主页: [cs224r.stanford.edu](https://cs224r.stanford.edu)

项目主页: <https://github.com/hqhq1025/ai-course-notes>

# 目录

|          |                               |          |
|----------|-------------------------------|----------|
| <b>1</b> | <b>从模仿学习到强化学习</b>             | <b>2</b> |
| 1.1      | Online RL 的基本流程 . . . . .     | 2        |
| 1.2      | 本章小结 . . . . .                | 2        |
| <b>2</b> | <b>策略梯度的推导</b>                | <b>2</b> |
| 2.1      | RL 目标函数 . . . . .             | 2        |
| 2.2      | Log-Gradient Trick . . . . .  | 2        |
| 2.3      | REINFORCE 算法 . . . . .        | 3        |
| 2.4      | 本章小结 . . . . .                | 3        |
| <b>3</b> | <b>直觉理解与改进</b>                | <b>3</b> |
| 3.1      | 梯度的含义 . . . . .               | 3        |
| 3.2      | 因果性改进: Reward-to-Go . . . . . | 3        |
| 3.3      | Baseline 改进 . . . . .         | 3        |
| 3.4      | 实现策略梯度 . . . . .              | 4        |
| 3.5      | 本章小结 . . . . .                | 4        |
| <b>4</b> | <b>Off-Policy 策略梯度</b>        | <b>4</b> |
| 4.1      | On-Policy 的局限 . . . . .       | 4        |
| 4.2      | Importance Sampling . . . . . | 4        |
| 4.3      | 本章小结 . . . . .                | 4        |
| <b>5</b> | <b>总结与延伸</b>                  | <b>4</b> |
| 5.1      | 拓展阅读 . . . . .                | 5        |

# 1 从模仿学习到强化学习

上一讲介绍了模仿学习：给定专家演示，通过监督学习来学习策略。模仿学习的局限是：(1) 不能超越演示者；(2) 没有”练习”和自我改进的机制。

## Offline vs. Online

- **Offline**: 仅使用已有数据集，不需要从学习到的策略收集新数据。
- **Online**: 使用学习到的策略收集新数据。

策略梯度是我们介绍的第一个 online RL 算法。

## 1.1 Online RL 的基本流程

1. 初始化策略（随机、用模仿学习、或用启发式方法）
2. 运行策略收集一批数据
3. 使用数据改进策略
4. 重复步骤 2-3

## 1.2 本章小结

策略梯度方法通过直接优化 RL 目标函数来改进策略，克服了模仿学习的局限性。

# 2 策略梯度的推导

## 2.1 RL 目标函数

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[ \sum_t r(s_t, a_t) \right]$$

## 2.2 Log-Gradient Trick

利用恒等式  $\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$ ，可以将梯度写成：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) \cdot r(\tau)]$$

### 关键简化

由于  $\log p_{\theta}(\tau) = \log p(s_1) + \sum_t [\log \pi_{\theta}(a_t | s_t) + \log p(s_{t+1} | s_t, a_t)]$ ，其中只有  $\log \pi_{\theta}(a_t | s_t)$  依赖于  $\theta$ 。因此：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[ \left( \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) \cdot r(\tau) \right]$$

无需知道环境动力学  $p(s_{t+1} | s_t, a_t)$ 。

## 2.3 REINFORCE 算法

用采样来近似期望：

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left( \sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \cdot r(\tau_i)$$

完整算法：(1) 从  $\pi_{\theta}$  采样  $N$  条轨迹；(2) 计算梯度；(3) 更新  $\theta$ 。

## 2.4 本章小结

策略梯度通过 log-gradient trick 绕开了对环境模型的需求，直接从采样轨迹估计目标函数的梯度。REINFORCE 是最基础的策略梯度算法。

# 3 直觉理解与改进

## 3.1 梯度的含义

策略梯度本质上就是**按奖励加权的模仿学习梯度**：

- 高奖励轨迹中的动作：增加其似然。
- 低奖励（或负奖励）轨迹中的动作：降低其似然。

这是“试错学习”的数学形式化：多做好事，少做坏事。

## 3.2 因果性改进：Reward-to-Go

时刻  $t$  的行为不应该受到  $t' < t$  时刻奖励的影响。因此用**未来奖励之和**（reward-to-go）代替总奖励：

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \left( \sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) \right)$$

## 3.3 Baseline 改进

### 对奖励尺度敏感

如果所有奖励都是正的，梯度会增加所有动作的似然——即使有些动作只是“不太差”。这导致梯度噪声很大。

解决方案：减去一个 **baseline**  $b$ （通常取平均奖励）：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) \cdot (r(\tau) - b)]$$

关键性质：减去常数 baseline **不会引入偏差**（梯度的期望不变），但可以**降低方差**。减去平均奖励后，低于平均的行为会被惩罚，高于平均的行为会被鼓励。

### 3.4 实现策略梯度

实现时不需要逐个计算  $N \times T$  次反向传播。可以构造**代理目标函数** (surrogate objective):

$$\tilde{J}(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \log \pi_{\theta}(a_{i,t} | s_{i,t}) \left( \sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) - b \right)$$

对  $\tilde{J}$  求梯度即可得到策略梯度，本质上是**按奖励加权的最大似然**。

### 3.5 本章小结

策略梯度的核心直觉是”做更多好事，少做坏事”。因果性 (reward-to-go) 和 baseline 是两个关键改进，能降低梯度方差。

## 4 Off-Policy 策略梯度

### 4.1 On-Policy 的局限

Vanilla policy gradient 是完全 on-policy 的：每次梯度更新后，之前的数据就不能再用了。这意味着**每个梯度步都需要重新收集数据**，效率很低。

### 4.2 Importance Sampling

可以用 importance weights 来复用旧策略  $\pi_{\theta}$  收集的数据来更新新策略  $\pi_{\theta'}$ ：

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_{\theta}(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \left( \sum_{t'=t}^T r(s_{i,t'}, a_{i,t'}) - b \right)$$

#### Importance Sampling 不是万能的

当新旧策略差异很大时，importance weights 的方差会非常大，使得估计不可靠。因此 importance sampling 只适用于策略**非常接近**的情况。

### 4.3 本章小结

Vanilla policy gradient 是 on-policy 的，数据利用率低。Importance sampling 可以实现一定程度的 off-policy 更新，但受限于新旧策略的相似度。

## 5 总结与延伸

1. **策略梯度**通过 log-gradient trick 直接优化 RL 目标。
2. 核心直觉：增加高奖励行为的概率，降低低奖励行为的概率。
3. **Reward-to-go** 和 **baseline** 是降低方差的关键技巧。
4. 即使有这些改进，策略梯度仍然是**高方差**的，在奖励稀疏或 batch 较小时尤其明显。
5. 策略梯度是 on-policy 的，数据效率低；importance sampling 提供了有限的 off-policy 能力。
6. 下一讲将介绍 Actor-Critic 方法，通过学习价值函数来进一步降低方差。

## 5.1 拓展阅读

- Williams, *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning* (REINFORCE 原文, 1992)。
- Schulman et al., *High-Dimensional Continuous Control Using Generalized Advantage Estimation*。
- Sergey Levine, *CS285 Lecture on Policy Gradients*: 策略梯度推导的另一个优秀参考。